Coral by Zoorik

Built for **Azure and AWS**

Supported on **AKS and EKS**

Compatible with your native **Karpenter, NAP and Autoscaler**

FOR PLATFORM AND INFRASTRUCTURE TEAMS

Coral FAQ

Short answers to the questions platform teams ask us most about how Coral sizes workloads, removes spare nodes and works with your autoscaler.

- **Pod sizing is opt-in**

  Coral only recommends until you turn on **"Automatically optimize"** for a policy or a workload.

- **Node optimization is on from install**

  Removing spare nodes starts at install, through node optimization and the Evictor (which empties nodes for your autoscaler to delete). Switch both off, put the Evictor in dry-run, or limit them to chosen node pools.

Numbers are product defaults. Answers describe Coral as released in October 2026.

01	How Coral sizes workloads	Q1–8	02	Safety, memory and rollback	Q9–15
03	Stateful and single-replica workloads	Q16–17	04	Working with HPA	Q18–21
05	Nodes and spare capacity	Q22–25	06	Autoscalers and traffic surges	Q26–27
07	Getting started	Q28			

SECTION 1

How Coral sizes workloads

1 Is the recommendation based on Average, P95, P99, Max, or a mix?

- **CPU:** a percentile of the last 3 days (P80 by default), plus 10% headroom.
- With a CPU-based HPA, the CPU request comes from replaying your HPA instead (question 19).
- **Memory:** the peak of the last 8 days, plus 10% headroom, because running out of memory kills the container.
- During a surge or CPU pressure, the CPU target is raised for a while, up to P99.
- You can change the target per workload or in a custom policy, up to P99.9 or max.

2 What does each Coral policy do, and which one does a workload get?

- A policy sets how Coral sizes a workload's CPU and memory requests. Six come built in.
- **CPU:** Balanced uses the 80th percentile of the last 3 days plus 10%, Stability the 90th plus 15%, Cost savings the 70th plus 10%, and Burstable the 60th percentile of the last 8 days plus 25%.
- **Memory:** the highest use of the last 8 days, plus 10 to 20%.
- Resiliency sizes like Balanced and keeps limits equal to requests on Guaranteed pods. Read-only shows recommendations and never changes anything.
- By default, StatefulSets get Resiliency, Jobs and CronJobs get Burstable, the cluster's own add-ons get Read-only, and everything else gets Balanced. You can assign another policy to any of your workloads, or create your own.

3 How much history is needed before the first recommendation?

- A first recommendation appears within the first hour.
- It is trusted after 24 hours of data. Jobs and CronJobs need 3 runs.
- The window then grows to 3 days for CPU and 8 days for memory.

4 CPU is 100m at night and 2000m from 10 AM to 2 PM. What does Coral recommend?

- Without an HPA, the base request sits near the night level: the 4 busy hours are only 17% of the day, and P80 leaves out the busiest 20%.
- At 10 AM Coral detects the surge within minutes and raises the request. It drops back a few hours after the peak.
- With a CPU-based HPA, Coral sizes the request by replaying load through your HPA instead (question 19).
- To reserve 2000m all day, use the Stability policy or set a CPU minimum.

5 A workload is idle for 6 days and busy every Monday. Will Coral cut resources before Monday?

- **Memory: no.** Coral keeps last Monday's peak, because it looks back 8 days (unless more than about 10% of the data is missing).
- **CPU: yes.** A short Monday peak doesn't hold the CPU request up, so it returns to the idle level after the peak.
- On Monday Coral starts raising CPU within minutes. To be ready from the first minute, set a CPU minimum.

6 How are workloads handled that spike only at month-end or quarter-end?

- Coral looks back at most 8 days, so it doesn't remember last month's peak.
- When the spike comes, Coral raises CPU within minutes, and raises memory after any OOM kill.
- For known month-end workloads, set minimums or use a more conservative policy.

7 How does Coral learn Black Friday, Diwali or holiday traffic patterns?

- It doesn't forecast or use a calendar. It reacts to live usage.
- Your HPA adds pods, your autoscaler adds nodes, and Coral raises requests that run hot.
- Before a known event, set minimums on critical workloads.
- For the event window, pause node optimization and the Evictor.

8 A workload needs 4 CPUs during startup but only 500m afterwards. How is it handled?

- By default Coral ignores the first 2 minutes of CPU after a container starts, so the request ends up near 500m.
- Startup can still go above the request, using free CPU on the node, because Coral doesn't lower CPU limits on regular pods.
- If startup must be fast, set a CPU minimum or use a policy that counts startup CPU.
- Guaranteed StatefulSet pods and workloads with a CPU-based HPA are sized differently.

SECTION 2

Safety, memory and rollback

9 How does Coral decide a change is safe to apply automatically?

- Only workloads with "Automatically optimize" on are changed. Zoorik's own namespace is never changed.
- Through a policy, cuts are capped in the first 24 hours of data (question 11). After that Coral needs at least 90% of the expected samples.
- Every change respects your minimums, LimitRanges and quotas. Routine changes under 10% are skipped.
- Pods are resized in place where possible. Otherwise they are replaced one at a time, within your PDBs.

10 How does a change reach the pods of a Deployment, StatefulSet, DaemonSet or Job?

- New pods always start at the new size. Running pods are resized in place where the node allows it (Kubernetes 1.33+).
- **Deployment:** otherwise pods are replaced one at a time, each after the last is Ready, within your PodDisruptionBudgets.
- **StatefulSet:** one pod at a time, highest ordinal first, and never an extra pod.
- **DaemonSet:** one node at a time. **Job and CronJob:** a running job is left alone, and the next run gets the new size.
- Pods with no controller, and ReplicaSets on their own, aren't resized.

11 Is there a warm-up period before automation?

- For pod sizing, yes. When you turn it on through a policy, cuts in the first 24 hours of data are capped at 10%, then 25%, then 35%.
- Workloads that already have 24 hours of good data get the full recommendation straight away.
- Turning on a single workload directly skips this warm-up.
- Node optimization has no warm-up. It starts at install.

12 Memory spikes can cause OOM kills. How much safety buffer is kept?

- Memory is set from the highest usage of the last 8 days, plus 10% (15 to 20% on some other policies).
- Coral never lowers a memory limit on regular pods, so a container with a limit is OOM-killed at the same point.
- A container with no memory limit can be evicted sooner if the node runs short of memory.
- If an OOM kill still happens, Coral raises memory straight away (next question).

13 What happens if an optimized pod starts getting OOM-killed?

- Coral raises that container's memory by 20% per OOM kill (at least 100 MiB), straight away.
- Crash-looping pods are recreated at the new size, and the extra margin fades over 24 hours.
- Raises stop at 2.5× the base size. If the kills look like a memory leak, Coral stops raising and holds that cap for 24 hours.

14 If optimization causes performance degradation, how is it rolled back automatically?

- OOM kills, failing probes and CPU pressure make Coral raise requests back up automatically.
- If a pod can't be scheduled at its new size, Coral caps the request to fit.
- A replacement pod that isn't Ready within 10 minutes goes back to its original size (except in StatefulSets).
- Coral doesn't read latency or error rates.
- To roll back by hand, turn automation off. New pods start at your manifest's size; running pods keep theirs until they restart.

15 Does Coral change my Git or GitOps manifests?

- No. Coral never edits Git or your workload's template. It sets sizes on pods as they're created, or resizes running pods in place, so Argo CD and Flux see no drift.
- The one change Coral makes to a Deployment, a temporary second replica for a single-replica workload, is skipped when Argo CD or Flux own the replica count.
- A CPU or memory limit in your manifest doesn't cap Coral. Where it sits below the new request, the pod's limit is raised: to 100× the request for CPU, and 1.5× (at least 128 MiB) for memory. Without a limit, Coral adds none unless the namespace requires one.
- What does cap it: a maximum in a policy or in the workload's Coral annotation (which can live in Git), the namespace's LimitRange, and its ResourceQuota.

SECTION 3

Stateful and single-replica workloads

16 How are databases, Kafka, Elasticsearch, Redis and StatefulSets optimized?

- Coral changes only CPU and memory, never app settings such as JVM heap or Redis maxmemory.
- StatefulSet pods are resized in place, or replaced one at a time, highest ordinal first, within the PDB.
- When a node is removed, a StatefulSet pod gets a short restart within its PDB. Nodes with local data or do-not-disrupt pods are kept.
- For a careful start, put databases on the Read-only policy and mark them do-not-disrupt.

17 Can optimization happen without downtime for applications that have only one replica?

- Usually, yes. Coral resizes the pod in place (Kubernetes 1.33+), or starts a new pod and waits until it's Ready before removing the old one.
- Where that isn't safe (for example Recreate strategy or a ReadWriteOnce volume), the new size waits for the pod's next natural restart.
- When a node is removed, a pod whose replacement can't start first gets a short restart. Mark it do-not-disrupt to prevent that.

SECTION 4

Working with HPA

18 If the HPA wants to scale up while Coral wants to reduce CPU requests, which one wins?

- There is no conflict. The HPA decides how many pods run, and Coral decides how big each pod is.
- While the HPA is scaling and below its max, Coral holds back its surge and CPU-pressure raises (OOM raises still apply), and pauses pod replacements until every pod is Ready.
- For CPU-based HPAs, Coral picks the request by replaying load through your HPA's own settings (next question).

19 Could reducing CPU requests make the HPA scale more aggressively?

- It could, because HPA utilization is usage divided by request.
- So for workloads with a CPU-based HPA, Coral replays the last 3 days of load through your HPA's own settings.
- A smaller request wins only if its saving beats the cost of unserved load and extra scaling.

20 Does optimization ever modify HPA minReplicas or maxReplicas?

- Only on HPAs Coral manages: ones it creates for safe Deployments, with minReplicas of 2 or more, and ones you created by hand that it takes over.
- On an HPA it takes over, Coral keeps minReplicas, only raises maxReplicas, and restores your values when it lets go.
- An HPA that Coral creates can scale a steady Deployment down to its minimum.
- It never changes min or max on HPAs from Helm, Argo CD, Flux, KEDA or kubectl apply.

21 What happens when demand goes past the maxReplicas set in Git?

- Your maxReplicas stays. Coral never changes min or max on an HPA managed from Git.
- When the HPA reaches its max, Coral records it and recalculates. For a CPU-based HPA, its sizing replays the last 3 days of load through your HPA's settings, max included, so when the HPA can't add pods, each pod gets more CPU instead.
- Coral's surge and CPU-pressure raises, held back while the HPA is still scaling, switch on once it is at its max.
- Each raise still stays within your LimitRange and ResourceQuota.

SECTION 5

Nodes and spare capacity

22 If workload optimization reduces requests, how soon are nodes consolidated and removed?

- Within minutes of the running pods carrying the smaller requests. Coral checks every 5 minutes and drains nodes gradually, never in bulk.
- An emptied node waits 5 minutes (you can change this), then your autoscaler deletes it. Coral deletes it itself only in fixed-size pools that nothing else manages.
- Pools that Karpenter or NAP already consolidates are left to them.

23 How do you make sure node consolidation doesn't cause scheduling failures later?

- Before draining, Coral simulates the scheduler on the live cluster. Every pod must fit elsewhere, with its affinity, taints, spread, ports and zones.
- It respects PDBs, do-not-disrupt and your autoscaler's minimum pool size.
- Room for future growth is your autoscaler's job.
- Known gap: per-node disk-attach limits aren't checked yet.

24 How much spare capacity is kept for sudden workload growth?

- Coral keeps no idle spare nodes. The headroom sits inside each pod's request.
- Sudden growth is handled by your autoscaler adding nodes.
- To keep warm spare capacity, run a low-priority placeholder deployment. Put it on the Read-only policy so Coral doesn't shrink it.

25 During cluster upgrades, is optimization paused or does it continue?

- It continues, but steps around the upgrade. It skips cordoned, NotReady or deleting nodes, and never drains a node someone else cordoned.
- We recommend pausing node optimization and the Evictor during the upgrade window.

SECTION 6

Autoscalers and traffic surges

26 How does Coral interact with Cluster Autoscaler or Karpenter decisions?

- Coral never changes your autoscaler's settings and never adds nodes for demand.
- Karpenter, NAP or the Cluster Autoscaler deletes the nodes Coral empties, within your autoscaler's minimums.
- Pools that Karpenter or NAP already consolidates are left to them. Coral deletes VMs itself only in fixed-size pools that nothing else manages.

27 After node and workload optimization move us from larger machines to smaller ones, how do we get back to bigger machines in a sudden surge?

- Coral never adds capacity for demand. Scale-up belongs to your Karpenter, NAP or Cluster Autoscaler, and Coral doesn't change or fight it.
- In a surge, the HPA adds pods, pending pods trigger your autoscaler, and Karpenter or NAP can pick bigger machines.
- If you have no autoscaler, we set up NAP (AKS) or Karpenter (EKS) during onboarding, with your approval.
- If one can't be set up (for example AKS with kubenet networking), nothing adds nodes until an autoscaler exists.
- Coral moves workloads to smaller machine types only with Karpenter or NAP. With only the Cluster Autoscaler, Coral removes nodes but never changes their size.

SECTION 7

Getting started

28 Can optimization be restricted to one namespace or application first?

- Yes. Create a policy with automation on and assign it to a namespace or by labels. Everything else stays recommendation-only.
- That policy can also create HPAs. Turn HPA optimization off in it if you want sizing only.
- Node optimization is scoped by node pool, not namespace. Enable only the pools you want, and start with the Evictor in dry-run.

Find your own number.

Run the free assessment on your own cluster.

zoorik.com/assessment →

Talk to the builders.

Pick a time at cal.com/zoorik/talk, or write to founders@zoorik.com.